(assuming there's an existing variable called sam that references an appropriate object which we have defined elsewhere).

When we talk about methods being sent, we often say that we send a *message* to the object. The message contains the method's name, along with any arguments the method may expect. The object responds to the message by invoking the method with that name. This idea of expressing method calls in the form of messages to objects comes from the programming language Smalltalk. When an object receives a message, it looks into its own class for a corresponding method. If found, that method is executed. If the method *isn't* found, Ruby goes off to look for it—we'll get to where Ruby looks in Method Lookup, on page 113.

It's worth noting here a major difference between Ruby and other object-oriented languages. In Java, for example, you'd find the absolute value of some number by calling a separate function and passing in that number. You could write this:

```
num = Math.abs(num);     // Java code
```

In Ruby, the ability to determine an absolute value is built into the numbers class which takes care of the details internally. You simply send the message abs to a number object and let it do the work:

```
num = -1234        # => -1234
positive = num.abs # => 1234
```

The same applies to all Ruby objects. In Python you'd write len(name), but in Ruby it would be name.length, and so on. This is what we mean when we say that Ruby is a pure object-oriented language with no basic types.

Some Basic Ruby

Not many people like to read heaps of boring syntax rules when they're picking up a new language, so we're going to cheat. In this section, we'll hit the highlights—the stuff you'll *need* to know if you're going to write Ruby programs. Later, in Part IV, Ruby Language Reference, on page 429, we'll go into all the gory details.

Let's start with a simple Ruby program. We'll write a method that returns a personalized greeting. We'll then invoke that method a couple of times:

intro/hello1.rb

```
def say_hello_goodbye(name)
  result = "I don't know why you say goodbye, " + name + ", I say hello"
  return result
end

# call the method
puts say_hello_goodbye("John")
puts say_hello_goodbye("Paul")
```

produces:

```
I don't know why you say goodbye, John, I say hello
I don't know why you say goodbye, Paul, I say hello
```

As the example shows, Ruby syntax is uncluttered. You don't need semicolons at the ends of statements as long as you put each statement on a separate line. Ruby comments start with a # character and run to the end of the line. Code layout is up to you; indentation is